

# Plan profesional de migración - Gym System

---

Migración completa de PHP MVC a NestJS + Flutter + IA

Preparado para el proyecto Gym System

2026-07-02

---

## Índice

1. Resumen ejecutivo
  - 1.1 Estado actual del proyecto (2026-07-02)
2. Diagnóstico del sistema actual
  - 2.1 Tipo de arquitectura actual
  - 2.2 Base de datos actual
  - 2.3 Módulos detectados
  - 2.4 Hallazgos técnicos positivos
  - 2.5 Riesgos técnicos actuales
3. Objetivo de la migración
  - 3.1 Objetivo general
  - 3.2 Objetivos específicos
4. Arquitectura objetivo
  - 4.1 Arquitectura general
  - 4.2 Principio central
5. Decisión sobre base de datos
  - 5.1 Recomendación inicial
  - 5.2 Recomendación futura
  - 5.3 ORM recomendado
  - 5.4 Estrategia implementada (Fase 01)
6. Estructura recomendada de NestJS
7. Estructura recomendada de Flutter
  - 7.1 App cliente
  - 7.2 Panel administrativo
8. Plan completo de migración por fases
  - 8.0 Correcciones obligatorias (no replicar bugs PHP)
  - 8.1 Criterios para avanzar de fase

## Fase 00 - Diagnóstico completo del sistema PHP

- Objetivo

- Entregables

- Criterio de finalización

## Fase 01 - Backend base NestJS

- Objetivo

- Módulos creados

- Endpoints

- Base de datos

## Fase 02 - Auth, usuarios y roles

- Objetivo

- Módulos creados

- Roles

- Endpoints principales

- Cambios BD

## Fase 03 - Socios, planes y membresías

- Objetivo

- Módulos creados

- Endpoints principales

- Reglas clave

- Cambios BD

## Fase 04 - Asistencias y QR

- Objetivo

- Módulos creados

- Endpoints principales

- Reglas clave

- Cambios BD

## Fase 05 - Progreso físico y rutinas

- Objetivo

- Módulos creados

- Endpoints principales

- Reglas clave

## Fase 06 - Inventario y productos

- Objetivo

- Módulos creados

- Endpoints principales

- Cambios BD

## Fase 07 - POS, ventas y caja

- Objetivo

- Endpoints ampliados

- Reglas clave

## Fase 08 - Reportes y exportaciones

- Objetivo

Módulos creados/ampliados

Endpoints principales

#### Fase 09 - Facturación electrónica SRI (Ecuador)

Objetivo

Módulos creados

Endpoints principales

Pendientes (slice 3)

Principio

#### Fase 10 - Flutter app cliente

Objetivo

App socio - pantallas mínimas

Panel administrativo - módulos

Seguridad Flutter

Dependencias backend

#### Fase 11 - IA Gemini y WebSockets

Objetivo

Arquitectura obligatoria

Herramientas internas sugeridas

Endpoints planificados

WebSockets (eventos)

Límites de seguridad

Orden interno recomendado

#### Fase 12 - Cierre de migración

Objetivo

Actividades

Checklist de corte

Orden migración de datos

### 9. Mapa de módulos PHP a NestJS

### 10. Orden recomendado de desarrollo

### 11. MVP recomendado

MVP backend - [OK] Implementado (Fases 02–05)

MVP Flutter cliente - [PENDIENTE] Fase 10

MVP administrativo - [PENDIENTE] Fase 10

### 12. Reglas de negocio críticas a conservar

Socios

Membresías

Asistencias

Ventas

Caja

SRI

### 13. Prompt guía para Cursor - fase inicial (Fase 00)

### 14. Prompt guía para Cursor - backend base (Fase 01)

15. Prompt guía para Cursor - IA y WebSockets (Fase 11)
16. Criterios de calidad final
17. Conclusión profesional

# 1. Resumen ejecutivo

Este documento presenta un plan completo, profesional y paso a paso para migrar el sistema actual de gimnasio desarrollado en PHP hacia una arquitectura moderna basada en:

- **Backend principal:** NestJS.
- **Frontend móvil y web:** Flutter.
- **Base de datos central:** MySQL inicialmente, con opción futura de PostgreSQL.
- **IA para clientes:** módulo de asistente inteligente usando Gemini u otro proveedor compatible.
- **Tiempo real:** WebSockets para chat IA, notificaciones y eventos en vivo.
- **Facturación electrónica:** módulo aislado para SRI Ecuador.

La recomendación técnica es **no reescribir todo de forma improvisada**. El sistema PHP actual debe ser tratado como una fuente de reglas de negocio y datos históricos. La nueva plataforma debe construirse por fases, migrando primero lo crítico: autenticación, socios, membresías, asistencias y progreso físico. Después se migran inventario, POS, caja, reportes y facturación electrónica.

La meta final es que **toda la lógica de negocio viva en NestJS**, mientras Flutter consume la API. PHP quedará únicamente como referencia temporal hasta apagarlo completamente.

## 1.1 Estado actual del proyecto (2026-07-02)

Documentación oficial por fase en docs/fases/. Resumen de avance:

| Fase | Nombre                     | Estado          | Documento                                  |
|------|----------------------------|-----------------|--------------------------------------------|
| 00   | Diagnóstico PHP            | [OK] Completada | docs/fases/fase-00-diagnostico.md          |
| 01   | Backend base NestJS        | [OK] Completada | docs/fases/fase-01-backend-base.md         |
| 02   | Auth, usuarios, roles      | [OK] Completada | docs/fases/fase-02-auth-usuarios-roles.md  |
| 03   | Socios, planes, membresías | [OK] Completada | docs/fases/fase-03-socios-membresias.md    |
| 04   | Asistencias y QR           | [OK] Completada | docs/fases/fase-04-asistencias-qr.md       |
| 05   | Progreso físico y rutinas  | [OK] Completada | docs/fases/fase-05-progreso-rutinas.md     |
| 06   | Inventario y productos     | [OK] Completada | docs/fases/fase-06-inventario-productos.md |
| 07   | POS, ventas y caja         | [OK] Completada | docs/fases/fase-07-pos-ventas-caja.md      |

| Fase | Nombre                   | Estado                                  | Documento                                  |
|------|--------------------------|-----------------------------------------|--------------------------------------------|
| 08   | Reportes y exportaciones | [OK] Aprobada                           | docs/fases/fase-08-reportes.md             |
| 09   | Facturación SRI Ecuador  | [EN CURSO] En curso (slice 3 pendiente) | docs/fases/fase-09-facturacion-sri.md      |
| 10   | Flutter app cliente      | [PENDIENTE] Pendiente                   | docs/fases/fase-10-flutter-app-cliente.md  |
| 11   | IA Gemini + WebSockets   | [PENDIENTE] Pendiente                   | docs/fases/fase-11-ia-websockets-gemini.md |
| 12   | Cierre de migración      | [PENDIENTE] Pendiente                   | docs/fases/fase-12-cierre-migracion.md     |

**Fase activa:** 09 - Facturación SRI (emisión operativa; pendiente RIDE PDF y email).

**Regla de avance:** cada fase requiere documentación, pruebas, bitácora y **aprobación explícita** antes de continuar. Ver docs/04-plan-migracion-fases.md.

## 2. Diagnóstico del sistema actual

### 2.1 Tipo de arquitectura actual

El repositorio actual es un sistema **PHP MVC tradicional**. Tiene un archivo de entrada en `public/index.php` que interpreta la URL, arma dinámicamente el controlador, método y parámetros, y luego ejecuta el controlador correspondiente.

Esto significa que el sistema funciona, pero tiene limitaciones para una aplicación moderna porque:

- La lógica está acoplada a sesiones PHP.
- Las vistas HTML están mezcladas con el flujo del backend.
- La API no está separada del frontend.
- No existe una capa formal de contratos REST.
- No hay DTOs, guards, interceptores ni documentación automática tipo Swagger.
- La seguridad depende de validaciones manuales por controlador.

### 2.2 Base de datos actual

La conexión actual usa MySQL con la base `ec_gym_system`. Las credenciales aparecen directamente dentro del código (`localhost`, `root`, contraseña vacía). Esto es aceptable solo para desarrollo local, pero no para producción.

Problemas detectados:

- Configuración quemada en código.
- Falta de `.env` real para entornos.

- Falta de migraciones versionadas modernas.
- Falta de validación formal del esquema.
- Riesgo de inconsistencias si PHP y NestJS escriben al mismo tiempo sin estrategia.

## 2.3 Módulos detectados

El sistema actual contiene módulos importantes que deben migrarse cuidadosamente:

- Autenticación y usuarios.
- Roles: administrador, recepcionista, entrenador.
- Socios del gimnasio.
- Planes.
- Suscripciones o membresías.
- Asistencias.
- Carnet con QR.
- Progreso físico: medidas y rutinas.
- Inventario: categorías y productos.
- POS y ventas.
- Caja.
- Gastos.
- Reportes.
- Configuración del negocio.
- Notificaciones por WhatsApp.
- Facturación electrónica SRI Ecuador.

## 2.4 Hallazgos técnicos positivos

El sistema actual no debe descartarse sin análisis. Tiene puntos positivos:

- Ya usa `password_hash` y `password_verify` para contraseñas.
- Ya maneja roles básicos.
- Ya registra ventas usando transacciones en el modelo `Venta`.
- Ya descuenta stock al vender.
- Ya tiene progreso físico y rutinas, una base útil para la IA.
- Ya tiene flujo de facturación electrónica SRI.
- Ya genera reportes, tickets, carnets y comprobantes.

## 2.5 Riesgos técnicos actuales

Los riesgos principales son:

1. **Sesiones PHP:** no son ideales para Flutter ni APIs móviles.
2. **Credenciales en código:** deben pasar a variables de entorno.
3. **Admin por defecto:** existe riesgo si queda activo con contraseña simple.
4. **Controladores con demasiada lógica:** varias reglas deberían pasar a servicios.

5. **Subida de archivos poco controlada:** necesita validación fuerte de tipo, tamaño y extensión.
6. **Mezcla conceptual SUNAT/SRI:** se deben limpiar referencias fiscales no aplicables a Ecuador.
7. **Ausencia de pruebas automatizadas:** dificulta migrar sin romper.
8. **Carrito en sesión:** el POS debe migrar a una lógica persistente o controlada por API.
9. **Lógica duplicada:** varias verificaciones se repiten en controladores.
10. **Sin contrato API:** Flutter necesitará endpoints claros, versionados y documentados.

## 3. Objetivo de la migración

---

El objetivo no es solamente cambiar PHP por NestJS. El objetivo real es convertir el sistema en una **plataforma moderna de administración de gimnasio**.

### 3.1 Objetivo general

Migrar el sistema actual PHP a una arquitectura moderna centralizada en NestJS, con frontend Flutter para administración y app cliente, incorporando IA, WebSockets, seguridad profesional, documentación API, pruebas automatizadas y despliegue preparado para producción.

### 3.2 Objetivos específicos

- Centralizar toda la lógica de negocio en NestJS.
- Crear una API REST documentada con Swagger.
- Crear autenticación moderna con JWT y refresh tokens.
- Implementar roles y permisos granulares.
- Migrar socios, membresías, asistencias, inventario, ventas, caja y facturación.
- Crear una app Flutter para socios.
- Crear panel administrativo moderno en Flutter Web o Flutter Desktop/Web.
- Integrar IA para seguimiento físico, rutinas, asistencia y recomendaciones generales.
- Agregar WebSockets para chat IA y notificaciones en tiempo real.
- Mantener integridad histórica de la base de datos.
- Reducir riesgos mediante migración por fases.

## 4. Arquitectura objetivo

### 4.1 Arquitectura general

```
Flutter App Cliente
Flutter Admin / Web
    |
    v
NestJS API Gateway / Backend Modular
    |
    +-- Auth / Usuarios / Roles
    +-- Socios / Membresías / Planes
    +-- Asistencias / QR
    +-- Progreso físico / Rutinas
    +-- Inventario / POS / Caja
    +-- Reportes
    +-- Facturación SRI
    +-- IA Assistant / WebSockets
    +-- Notificaciones
    |
    v
Base de datos central
    |
    +-- Archivos / certificados / imágenes
    +-- Servicios externos: SRI, WhatsApp, Email, Gemini
```

### 4.2 Principio central

La regla principal será:

```
Flutter no decide reglas de negocio.
PHP deja de decidir reglas de negocio.
NestJS concentra toda la lógica.
```

Ejemplo correcto:

- Flutter pregunta: `GET /api/members/:id/membership`.
- NestJS calcula si la membresía está activa, vencida o próxima a vencer.
- Flutter solo muestra el resultado.

Ejemplo incorrecto:

- Flutter calcula fechas de vencimiento.
- PHP actualiza estados por separado.
- NestJS calcula otra cosa distinta.

Ese escenario genera errores e inconsistencias.

## 5. Decisión sobre base de datos

---

### 5.1 Recomendación inicial

Como el sistema actual ya usa MySQL, la recomendación es:

Mantener MySQL durante la primera migración.

Esto reduce riesgos porque:

- Los datos actuales ya están en MySQL.
- La migración es más rápida.
- Se evita cambiar demasiadas cosas al mismo tiempo.
- Se facilita comparar datos entre PHP y NestJS.

### 5.2 Recomendación futura

Cuando el sistema esté estable, se puede evaluar migrar a PostgreSQL si se requieren capacidades más avanzadas, pero no es obligatorio.

### 5.3 ORM recomendado

Se recomienda usar **Prisma** por estas razones:

- Es claro para modelar entidades.
- Tiene migraciones versionadas.
- Funciona bien con NestJS.
- Facilita tipado con TypeScript.
- Ayuda a documentar la estructura de datos.

Alternativa válida: TypeORM. Sin embargo, para un proyecto nuevo con Flutter + NestJS, Prisma suele ser más directo y mantenible.

### 5.4 Estrategia implementada (Fase 01)

En lugar de crear una base nueva desde cero, el proyecto **reutiliza** `ec_gym_system` importada del backup PHP (`npm run db:import-legacy`) e introspecciona con `prisma db pull`. Las tablas nuevas de NestJS (`auth_refresh_tokens`, `movimientos_inventario`, etc.) se agregan con `npm run db:push:legacy` sin reset de datos legacy.

# 6. Estructura recomendada de NestJS

---

```
src/  
  main.ts  
  app.module.ts  
  
  config/  
    env.validation.ts  
    app.config.ts  
    database.config.ts  
  
  common/  
    decorators/  
    filters/  
    guards/  
    interceptors/  
    pipes/  
    utils/  
    constants/  
  
  database/  
    prisma.service.ts  
    prisma.module.ts  
  
  auth/  
    auth.module.ts  
    auth.controller.ts  
    auth.service.ts  
    dto/  
    strategies/  
    guards/  
  
  users/  
  roles/  
  permissions/  
  
  members/  
  plans/  
  memberships/  
  attendance/  
  body-progress/  
  workout-routines/  
  
  inventory/  
  categories/  
  products/  
  sales/
```

```
cash-registers/  
expenses/
```

```
billing-sri/  
qr-access/  
reports/  
notifications/  
ai-assistant/  
websocket/  
files/  
audit-logs/
```

## 7. Estructura recomendada de Flutter

---

```
lib/  
  main.dart  
  app/  
    app.dart  
    router.dart  
    theme.dart  
  
  core/  
    api/  
    auth/  
    errors/  
    storage/  
    network/  
    widgets/  
    utils/  
  
  features/  
    auth/  
    home/  
    profile/  
    membership/  
    attendance/  
    progress/  
    routines/  
    ai_assistant/  
    notifications/  
    payments/  
    products/  
    admin_dashboard/  
    pos/  
    inventory/
```

### 7.1 App cliente

Debe incluir:

- Login del socio.
- Perfil del socio.
- Estado de membresía.
- QR personal de acceso.
- Historial de asistencias.
- Historial de pagos.

- Progreso físico.
- Rutinas asignadas.
- Chat con IA.
- Notificaciones.
- Alertas de vencimiento.

## 7.2 Panel administrativo

Debe incluir:

- Dashboard.
- Gestión de socios.
- Gestión de membresías.
- Planes.
- Asistencias.
- Inventario.
- POS.
- Caja.
- Reportes.
- Facturación.
- Configuración.
- Usuarios y roles.

## 8. Plan completo de migración por fases

El plan oficial del proyecto usa **13 fases numeradas del 00 al 12**. Cada fase tiene su documento detallado en `docs/fases/`. Este documento resume objetivos, entregables y estado; la bitácora vive en `docs/05-bitacora-migracion.md` y los endpoints en `docs/07-endpoints-api.md`.

### 8.0 Correcciones obligatorias (no replicar bugs PHP)

| ID  | Problema PHP                                     | Corrección NestJS                                                                |
|-----|--------------------------------------------------|----------------------------------------------------------------------------------|
| E01 | Caja sumaba suscripciones en lugar de ventas POS | <code>calculateSessionTotals()</code> desde <code>ventas</code>                  |
| E02 | Registrar asistencia sin revalidar membresía     | Validación en backend antes de cada registro                                     |
| E03 | Stock podía quedar negativo                      | <code>InventoryStockService</code> + movimientos auditados                       |
| E07 | Estados membresía inconsistentes                 | <code>membership-status.util.ts</code> - <code>effectiveStatus</code> en backend |
| E12 | Sin rol socio para app móvil                     | <code>POST /api/auth/member/login</code> + <code>JWT userType: member</code>     |

### 8.1 Criterios para avanzar de fase

- Documentación completada en `docs/fases/fase-XX-*.md`

- Bitácora actualizada en `docs/05-bitacora-migracion.md`
  - Pruebas en `docs/06-checklist-pruebas.md`
  - Endpoints en `docs/07-endpoints-api.md` (si aplica)
  - Auditoría de fase ( `npm run audit:phase-XX` ) en verde cuando exista
  - **Aprobación explícita del usuario**
- 

## Fase 00 - Diagnóstico completo del sistema PHP

**Estado:** [OK] Completada

**Documento:** `docs/fases/fase-00-diagnostico.md`

### Objetivo

Analizar el sistema PHP legacy ( `gym-system/` ) sin modificar código ni crear NestJS/Flutter.

### Entregables

- `docs/00-diagnostico-sistema-php.md`
- `docs/01-mapa-funcionalidades-actuales.md`
- `docs/02-mapa-base-datos-actual.md`
- Inventario de 80+ rutas, 20 controladores, 15 modelos, 17 tablas
- 13 errores documentados (E01-E13)

### Criterio de finalización

Sistema PHP documentado, respaldado y comparable contra la futura API NestJS.

---

## Fase 01 - Backend base NestJS

**Estado:** [OK] Completada

**Documento:** `docs/fases/fase-01-backend-base.md`

### Objetivo

Crear `backend-nest/` con base profesional NestJS + Prisma, sin modificar PHP.

### Módulos creados

- `DatabaseModule` + `PrismaService` (MariaDB, Prisma 7)
- `HealthController` (Terminus + ping DB)
- `HttpExceptionFilter` y `ResponseTransformInterceptor` globales
- Swagger en `/api/docs`

### Endpoints

```
GET /api/health
GET /api/docs
```

## Base de datos

- Importación legacy: `npm run db:import-legacy`
  - Introspección: 18 modelos Prisma desde `ec_gym_system`
- 

## Fase 02 - Auth, usuarios y roles

**Estado:** [OK] Completada

**Documento:** `docs/fases/fase-02-auth-usuarios-roles.md`

### Objetivo

Reemplazar sesiones PHP por **JWT Access + Refresh** con revocación server-side.

### Módulos creados

- `AuthModule`, `UsersModule`
- `JwtAuthGuard`, `JwtRefreshGuard`, `RolesGuard`
- `JwtAccessStrategy`, `JwtRefreshStrategy`
- `RefreshTokenService` (hash SHA-256, rotación)
- Decoradores: `@Public()`, `@Roles()`, `@CurrentUser()`

### Roles

`admin`, `repcionista`, `entrenador` (staff). Socio en Fase 03.

### Endpoints principales

```
POST /api/auth/login
POST /api/auth/refresh
POST /api/auth/logout
GET  /api/auth/me
GET  /api/users      (admin)
POST /api/users      (admin)
PATCH /api/users/:id (admin)
```

### Cambios BD

- Tabla nueva `auth_refresh_tokens` (solo NestJS)
- 

## Fase 03 - Socios, planes y membresías

**Estado:** [OK] Completada

**Documento:** `docs/fases/fase-03-socios-membresias.md`

### Objetivo

Migrar socios, planes y suscripciones con cálculo de estado de membresía en backend.

## Módulos creados

- `MembersModule`, `PlansModule`, `MembershipsModule`
- `membership-status.util.ts`

## Endpoints principales

|                |                                            |
|----------------|--------------------------------------------|
| GET/POST/PATCH | <code>/api/members</code>                  |
| GET            | <code>/api/members/:id/membership</code>   |
| GET/POST       | <code>/api/memberships</code>              |
| PATCH          | <code>/api/memberships/:id/cancel</code>   |
| GET/POST/PATCH | <code>/api/plans</code>                    |
| POST           | <code>/api/auth/member/login</code>        |
| POST           | <code>/api/members/:id/photo</code>        |
| GET            | <code>/api/memberships/export/excel</code> |

## Reglas clave

- DNI único; `effectiveStatus` calculado en backend
- `fecha_fin` = `fecha_inicio` + `plan.duracion_dias`
- Entrenador: lectura socios, sin membresías
- Estados: `activa`, `vencida` (cancelar = `vencida` en BD legacy)

## Cambios BD

- `socios.password` (login app móvil)
- `auth_refresh_tokens.memberId`

---

## Fase 04 - Asistencias y QR

**Estado:** [OK] Completada

**Documento:** `docs/fases/fase-04-asistencias-qr.md`

### Objetivo

Control de acceso, asistencias y QR con validación de membresía (corrige E02).

## Módulos creados

- `AttendanceModule`, `QrAccessModule`

## Endpoints principales

```
POST /api/attendance/validate
POST /api/attendance/register
POST /api/attendance/scan
POST /api/attendance/self      (socio)
GET  /api/attendance/today
GET  /api/attendance/report
POST /api/qr-access/validate
GET  /api/qr-access/members/:id/card
```

## Reglas clave

- Acceso: socio activo + membresía vigente
- Anti-duplicado mismo día (409 Conflict)
- Métodos: `manual`, `dni`, `qr`, `app`

## Cambios BD

- `asistencias.metodo_ingreso` (ENUM nullable)

---

## Fase 05 - Progreso físico y rutinas

**Estado:** [OK] Completada

**Documento:** `docs/fases/fase-05-progreso-rutinas.md`

## Objetivo

Migrar medidas corporales y rutinas; conservar historial para gráficos e IA.

## Módulos creados

- `BodyProgressModule`, `WorkoutRoutinesModule`
- `member-access.util.ts` (socio solo ve sus datos)

## Endpoints principales

```
GET/POST /api/body-progress/members/:memberId/measurements
GET       /api/body-progress/me
DELETE    /api/body-progress/measurements/:id
GET/POST  /api/workout-routines/members/:memberId
GET       /api/workout-routines/me/current
```

## Reglas clave

- Rutina: INSERT nueva versión (historial), no UPDATE
- Crear medidas/rutinas: admin y entrenador
- Recepcionista: solo lectura

## Fase 06 - Inventario y productos

**Estado:** [OK] Completada

**Documento:** docs/fases/fase-06-inventario-productos.md

### Objetivo

Categorías, productos, ajustes de stock y auditoría (corrige E03).

### Módulos creados

- categories/, products/, inventory/
- InventoryStockService
- Base POS: cash-registers/, sales/ (venta con descuento stock)

### Endpoints principales

```
GET/POST/PATCH  /api/categories
GET/POST/PATCH  /api/products
POST             /api/inventory/adjustments
GET              /api/inventory/movements
POST             /api/cash-registers/open
POST             /api/cash-registers/close
GET              /api/cash-registers/current
POST            /api/sales
```

### Cambios BD

- Tabla nueva `movimientos_inventario`

---

## Fase 07 - POS, ventas y caja

**Estado:** [OK] Completada

**Documento:** docs/fases/fase-07-pos-ventas-caja.md

### Objetivo

Completar flujo POS: historial, detalle, ticket, cierre con cuadre (corrige E01).

### Endpoints ampliados

```
GET  /api/sales
GET  /api/sales/:id
GET  /api/sales/:id/ticket
GET  /api/cash-registers/current/summary
GET  /api/cash-registers/history
POST /api/cash-registers/close
```

## Reglas clave

- No vender sin caja abierta
- Venta transaccional; descuento  $\leq$  total
- Totales caja desde ventas POS, no suscripciones
- Entrenador sin acceso POS

---

## Fase 08 - Reportes y exportaciones

**Estado:** [OK] Aprobada (2026-07-02)

**Documento:** docs/fases/fase-08-reportes.md

### Objetivo

Reportes financieros, export Excel/PDF, asistencias y ticket térmico.

### Módulos creados/ampliados

- reports/ - KPIs financieros
- attendance-export.service.ts, sales-ticket-export.service.ts
- pdf-buffer.util.ts, pdf-chart.util.ts

### Endpoints principales

```
GET /api/reports/financial/summary      (admin)
GET /api/reports/financial/movements   (admin)
GET /api/reports/financial/export/excel (admin)
GET /api/reports/financial/export/pdf   (admin)
GET /api/attendance/report/export/excel
GET /api/attendance/report/export/pdf
GET /api/sales/:id/ticket/export/pdf
```

---

## Fase 09 - Facturación electrónica SRI (Ecuador)

**Estado:** [EN CURSO] En curso - slice 1 y 2 completados; slice 3 pendiente

**Documento:** docs/fases/fase-09-facturacion-sri.md

### Objetivo

Migrar facturación SRI sin romper flujo fiscal: XML, firma, SOAP, logs.

### Módulos creados

- billing-sri/ - bandeja, emisión, reintento
- sri-xml-builder, sri-xml-signer, sri-soap-client, sri-tax-calculator

## Endpoints principales

```
GET /api/electronic-receipts
GET /api/electronic-receipts/:id
GET /api/electronic-receipts/:id/xml
GET /api/sri-config (admin)
POST /api/electronic-receipts/issue/membership/:membershipId
POST /api/electronic-receipts/issue/sale/:saleId
POST /api/electronic-receipts/:id/credit-note
POST /api/electronic-receipts/:id/retry
```

## Pendientes (slice 3)

- RIDE PDF
- Envío email al cliente
- Prueba con certificado P12 real en celcer

## Principio

POS registra la venta; `billing-sri` emite el documento fiscal de forma aislada.

---

## Fase 10 - Flutter app cliente

**Estado:** [PENDIENTE] Pendiente

**Documento:** docs/fases/fase-10-flutter-app-cliente.md

### Objetivo

App Flutter para socios y panel administrativo (Web/Desktop), consumiendo solo la API NestJS.

### App socio - pantallas mínimas

1. Login ( `POST /api/auth/member/login` )
2. Perfil y membresía
3. QR de acceso
4. Asistencias ( `/api/attendance/self` , historial)
5. Progreso físico ( `/api/body-progress/me` )
6. Rutina actual ( `/api/workout-routines/me/current` )
7. Notificaciones (cuando existan en Fase 11)

### Panel administrativo - módulos

- Dashboard, socios, membresías, asistencias
- Inventario, POS, caja, reportes, facturación SRI
- Usuarios y roles

### Seguridad Flutter

- Tokens en almacenamiento seguro

- Refresh automático; sin API Keys de Gemini en cliente
- Sin cálculo de reglas críticas en frontend

## Dependencias backend

Fases 02-05 mínimo para app socio; 06-09 para panel admin completo.

---

## Fase 11 - IA Gemini y WebSockets

**Estado:** [PENDIENTE] Pendiente

**Documento:** docs/fases/fase-11-ia-websockets-gemini.md

### Objetivo

Asistente IA para socios vía NestJS -> Gemini; WebSockets para chat y notificaciones.

### Arquitectura obligatoria

```
Flutter -> NestJS (ai-assistant) -> Gemini
      ↓
    Servicios internos (members, attendance, body-progress, etc.)
```

### Herramientas internas sugeridas

```
getMemberProfile()
getMembershipStatus()
getAttendanceSummary()
getBodyProgress()
getCurrentWorkoutRoutine()
saveAiNote()
```

### Endpoints planificados

```
POST /api/ai/chat
GET /api/ai/conversations
GET /api/ai/conversations/:id
```

### WebSockets (eventos)

```
client -> server: ai.message
server -> client: ai.typing | ai.response.chunk | ai.response.done
```

### Límites de seguridad

- No diagnóstico médico ni medicamentos
- Solo datos del usuario autenticado
- Rate limiting por usuario

## Orden interno recomendado

1. Chat por texto
  2. Herramientas internas
  3. WebSockets streaming
  4. Voz (fase posterior)
- 

## Fase 12 - Cierre de migración

**Estado:** [PENDIENTE] Pendiente

**Documento:** docs/fases/fase-12-cierre-migracion.md

### Objetivo

Corte operativo, validación integral y apagado controlado de PHP.

### Actividades

1. Migración de datos históricos validada (conteos y totales)
2. Pruebas automatizadas backend y Flutter
3. Seguridad: .env , JWT, rate limiting, auditoría, backups
4. Despliegue: development -> staging -> production
5. Capacitación usuarios internos
6. PHP en modo solo lectura -> redirección al nuevo sistema
7. Apagado PHP tras validación completa

### Checklist de corte

- Login staff y socio operativo
- Socios, membresías y asistencias correctos
- Productos, stock, caja y ventas cuadran con PHP
- Reportes y SRI validados en pruebas
- Flutter app funcional
- IA sin exposición de datos ajenos
- Backups automáticos activos

### Orden migración de datos

```
roles -> users -> members -> plans -> memberships -> attendance  
-> categories -> products -> cash sessions -> sales -> sale items  
-> billing documents -> progress -> routines
```

## 9. Mapa de módulos PHP a NestJS

| PHP actual                                  | Módulo NestJS                       | Fase  |
|---------------------------------------------|-------------------------------------|-------|
| AuthController / Usuario                    | auth , users                        | 02    |
| SociosController / Socio                    | members                             | 03    |
| PlanesController / Plan                     | plans                               | 03    |
| SuscripcionesController / Suscripción       | memberships                         | 03    |
| AsistenciaController / Asistencia           | attendance                          | 04    |
| CarnetController                            | qr-access                           | 04    |
| ProgresoController / Progreso               | body-progress , workout-routines    | 05    |
| InventarioController / Producto / Categoría | inventory , products , categories   | 06    |
| PosController / Venta                       | sales                               | 06-07 |
| CajaController / Caja                       | cash-registers                      | 06-07 |
| GastosController / Gasto                    | consultas en reports (CRUD futuro)  | 08    |
| ReportesController / Reporte                | reports                             | 08    |
| FacturacionElectronicaController            | billing-sri ( electronic-receipts ) | 09    |
| ConfiguracionController                     | sri-config (parcial)                | 09    |
| NotificacionesController                    | notifications (Fase 11)             | 11    |
| -                                           | ai-assistant , websocket            | 11    |

## 10. Orden recomendado de desarrollo

Orden oficial alineado con docs/fases/ y docs/04-plan-migracion-fases.md :

| #  | Fase            | Nombre                                | Estado |
|----|-----------------|---------------------------------------|--------|
| 00 | Diagnóstico PHP | Análisis legacy sin cambios           | [OK]   |
| 01 | Backend base    | NestJS + Prisma + health + Swagger    | [OK]   |
| 02 | Auth            | JWT Access/Refresh + usuarios + roles | [OK]   |
| 03 | Socios          | Members, plans, memberships           | [OK]   |

| #  | Fase        | Nombre                                  | Estado      |
|----|-------------|-----------------------------------------|-------------|
| 04 | Asistencias | Attendance + QR                         | [OK]        |
| 05 | Progreso    | Body progress + workout routines        | [OK]        |
| 06 | Inventario  | Categories, products, stock, venta base | [OK]        |
| 07 | POS/caja    | Historial ventas, cierre, ticket        | [OK]        |
| 08 | Reportes    | Financiero, Excel/PDF, exports          | [OK]        |
| 09 | SRI         | Facturación electrónica Ecuador         | [EN CURSO]  |
| 10 | Flutter     | App socio + panel admin                 | [PENDIENTE] |
| 11 | IA          | Gemini + WebSockets                     | [PENDIENTE] |
| 12 | Cierre      | Migración datos, pruebas, apagado PHP   | [PENDIENTE] |

**Nota:** El backend operativo (fases 00-08) precede a Flutter (10). La IA (11) depende de Flutter y progreso físico (05). SRI (09) se completa antes o en paralelo con Flutter según prioridad fiscal.

## 11. MVP recomendado

### MVP backend - [OK] Implementado (Fases 02-05)

- Auth JWT staff + socio ( /api/auth/login , /api/auth/member/login )
- Socios, planes, membresías con estado calculado
- Asistencias con validación de membresía y QR
- Progreso físico y rutinas

**Pendiente para MVP completo:** IA básica (Fase 11) y UI Flutter (Fase 10).

### MVP Flutter cliente - [PENDIENTE] Fase 10

- Login socio
- Perfil y membresía actual
- QR de acceso
- Asistencias y progreso
- Rutina vigente
- Chat IA (cuando exista Fase 11)

### MVP administrativo - [PENDIENTE] Fase 10

- Login admin/recepcionista

- Gestión de socios y membresías
- Registro de asistencia
- POS y caja (API lista en Fases 06-07)
- Reportes básicos (API lista en Fase 08)

## 12. Reglas de negocio críticas a conservar

---

### Socios

- Un socio tiene estado activo o inactivo.
- Un socio puede tener foto.
- El documento debe ser único.
- El socio puede tener acceso a la app.

### Membresías

- Una membresía activa da acceso.
- Una membresía vencida bloquea acceso.
- Se debe mantener historial.
- La renovación no debe borrar datos anteriores.

### Asistencias

- La asistencia se registra con fecha y hora.
- Solo debe registrarse si el socio puede acceder.
- Debe poder consultarse por fecha.

### Ventas

- No vender sin caja abierta.
- No vender sin stock.
- La venta descuenta stock.
- La venta suma a caja.
- La venta debe ser atómica.

### Caja

- Un usuario puede abrir caja.
- Se registra monto inicial.
- Se calculan ventas y gastos.
- Se calcula diferencia al cerrar.

### SRI

- El secuencial debe ser consistente.

- La clave de acceso debe generarse correctamente.
- El XML autorizado no debe alterarse.
- Se debe guardar log de envío y respuesta.

## 13. Prompt guía para Cursor - fase inicial (Fase 00)

---

Ejecutar solo FASE 00: Diagnóstico completo del sistema PHP.  
No crear NestJS, Flutter ni modificar funcionalidades.

Entrega obligatoria en docs/ (no crear archivos duplicados):

- docs/00-diagnostico-sistema-php.md
- docs/01-mapa-funcionalidades-actuales.md
- docs/02-mapa-base-datos-actual.md
- docs/fases/fase-00-diagnostico.md

Al terminar, esperar aprobación antes de Fase 01.

## 14. Prompt guía para Cursor - backend base (Fase 01)

---

Crea backend-nest/ con base NestJS profesional (Fase 01).

Requisitos:

1. TypeScript estricto, .env, validación de entorno
2. Prisma + MySQL/MariaDB (ec\_gym\_system legacy)
3. Swagger en /api/docs, GET /api/health
4. Filtro global de errores + interceptor de respuesta
5. Documentar en docs/fases/fase-01-backend-base.md

No implementar auth, inventario, POS ni SRI en esta fase.

## 15. Prompt guía para Cursor - IA y WebSockets (Fase 11)

---

Implementar módulo ai-assistant en NestJS (Fase 11).

Reglas:

1. Flutter -> NestJS -> Gemini (nunca API Key en cliente)
2. Herramientas internas: getMemberProfile, getMembershipStatus, getAttendanceSummary, getBodyProgress, getCurrentWorkoutRoutine
3. Chat por texto primero; WebSockets después
4. Documentar en docs/fases/fase-11-ia-websockets-gemini.md

## 16. Criterios de calidad final

---

El proyecto migrado debe cumplir:

- API documentada.
- Código modular.
- Lógica centralizada.
- Seguridad por roles.
- Pruebas automatizadas.
- Migración de datos validada.
- Flutter sin lógica crítica duplicada.
- IA conectada a datos reales.
- SRI aislado y auditable.
- Reportes correctos.
- Logs y auditoría.
- Backups.
- Despliegue reproducible.

## 17. Conclusión profesional

---

La migración está en ejecución y es viable. A julio de 2026 el **backend NestJS cubre fases 00-08** (auth, socios, asistencias, progreso, inventario, POS, reportes). La **Fase 09 (SRI)** está en curso. Flutter (10), IA (11) y cierre (12) están pendientes.

La ruta correcta - alineada con docs/fases/ - es:

1. Mantener PHP como referencia temporal (Fase 00 [OK]).
2. Construir NestJS como backend central (Fases 01–09).
3. Crear Flutter consumiendo solo la API (Fase 10).
4. Integrar IA y WebSockets sobre datos reales (Fase 11).
5. Migración de datos, pruebas finales y apagado PHP (Fase 12).

Documentación viva del proyecto:

- Plan por fases: docs/04-plan-migracion-fases.md
- Detalle por fase: docs/fases/fase-00 ... fase-12
- Bitácora: docs/05-bitacora-migracion.md
- Endpoints: docs/07-endpoints-api.md

Este enfoque reduce riesgos, corrige bugs del PHP legacy (E01–E03) y deja el sistema preparado para crecer como plataforma moderna de gestión de gimnasio.